Christopher Shoo

179 Followers About Follow

Deploying django application to a production server part II

Christopher Shoo May 24, 2018 · 5 min read

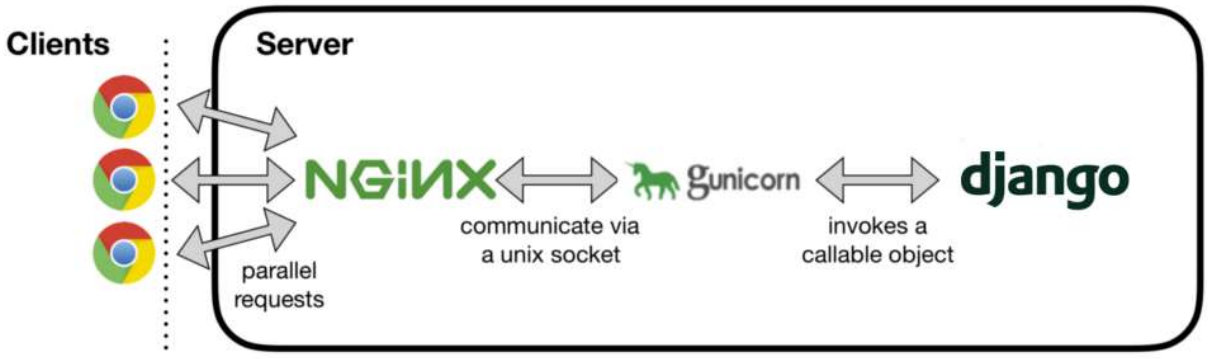
Welcome back, i'm glad you made it up to here, i promise there is going to be one hell of the happy ending on this one. So on the first part we had our app configured for deployment and we did some initial steps of the server configurations. If you don't know what i'm talking about, don't worry below is the link to the first part of this post.

On this part we are going to setup gunicorn first and lastly nginx so both of us can go to sleep.

I realised i say nginx and gunicorn a lot what are they anyway?

nginx is a web server that serves static files to the browser's request, static files such as html, css and images but if the request requires some logic that your python code executes, that's where gunicorn comes in. Gunicorn connects to your python code throung web server gateway interface (wsgi). The wsgi was designed to provide a way for the server to interact with your python code.

So how this configuration work is that, when a request from the browser comes, nginx checks if the request is for static or dynamic content. If the request is for static content nginx serves them to the browser but if the request is for dynamic content and requires some logic to be done, it passes the request to gunicorn and through wsgi gunicorn sends the request to your python code and python code will do the logic and provide the response. The response is sent back to nginx and hence the browser receives it.



We need nginx to serve static content and gunicorn to serve dynamic content, though that is not the only thing that nginx does, there are a lot more things that nginx does such as acting as a reverse proxy server, you can visit here *what is nginx?* to know more.

```
(env) $ pip install gunicorn
```

if the development server is still running kill it change directory to your project folder and run the following command to run your site with gunicorn

```
(env) $ gunicorn --bind 0.0.0.0:8800 AWESOME.wsgi:application
```

remember i told you that gunicorn uses wsgi to interact with your python code, now that is what we just did. We pointed gunicorn to the wsgi located under AWESOME folder in AWESOME_PROJECT directory (refer to my project directory structure on part one of this post). If you look you will see the file called `wsgi.py` and inside there will be a variable called application, that is where we pointed gunicorn to and now it's interacting with our python code. If you visit admin page you will notice that there will be no css, that is because nginx is not run yet.

kill gunicorn and exit the virtual environment with the following command

```
(env) $ deactivate
```

to finish up gunicorn setup we need to create gunicorn service file so that it runs when the system starts.

```
$ sudo nano /etc/systemd/system/gunicorn.service
```

paste the following configurations

```
After=network.target

[Service]
User=chris
Group=www-data
WorkingDirectory=/home/chris/webapp/AWESOME_PROJECT/
ExecStart=/home/chris/webapp/env/bin/gunicorn --access-logfile - --
workers 3 --bind unix:/home/chris/webapp/AWESOME_PROJECT/awesome.sock
AWESOME.wsgi:application

[Install]
WantedBy=multi-user.target
```

ofcourse you need to change some variables to match your project's location on your server and the user on your server.

on the `User` field change chris to your username, on the `WorkingDirectory` put the absolute path of your project's directory, also on the `ExecStrat` change the directories to match yours. You should make sure you change the variables right otherwise you will find yourself in trouble. Type the following command to start gunicorn.

```
$ sudo systemctl enable gunicorn.service
$ sudo systemctl start gunicorn.service
$ sudo systemctl status gunicorn.service
```

If you run into trouble, in most cases you will, just type the following command to see what went wrong

```
$ journalctl -u gunicorn
```

if you changed something restart gunicorn with the following command

```
$ sudo systemctl daemon-reload
$ sudo systemctl restart gunicorn
```

this documentation *gunicorn docs*. I encourage you to read that, it's a better way to know what you're doing.

configuring nginx

To configure nginx we need to create a config file that will tell nginx how to respond to a client's request. The file will live under Nginx's `sites-available` folder.

```
$ sudo nano /etc/nginx/sites-available/awesome
```

you can change awesome to other name, paste the following configurations

```
server {
        listen 80;
        server_name 127.0.0.1;
        location = /favicon.ico {access_log off;log_not_found off;}

        location /static/ {
          root /home/chris/webapp/AWESOME_PROJECT/AWESOME;
        }

        location /media/ {
          root /home/chris/webapp/AWESOME_PROJECT/AWESOME;
        }

        location / {
          include proxy_params;
          proxy_pass
http://unix:/home/chris/webapp/AWESOME_PROJECT/AWESOME.sock;
        }
      }
```

change the directories to match yours, the first line tells nginx to listen on port 80 and the second provides it with the servers ip address to respond to, put the ip address of your server or the dns address, each location variable tells nginx where to find files for different requests, the first tells nginx where to find favicon.ico and ignore erros if it could not find it, the second and third tells nginx where to find static and media files

following command

```
$ sudo ln -s /etc/nginx/sites-available/awesome /etc/nginx/sites-
enabled
```

that will add a link of our new configuration file and that way nginx will recognise it. Run the following command to test your configurations

```
$ sudo nginx -t
```

if everything worked fine restart nginx with the following command

```
$ sudo systemctl restart nginx
```

Before it's over we need to change our firewall rules and add port 80 because it is currently blocked. Type the following commands to do that

```
$ sudo ufw delete allow 8800
$ sudo ufw allow 'Nginx Full'
```

if everything worked untill now, you should be good to go. You can now go to your server ip or domain name to see your application.

Thank you for taking your time reading my posts. I appreciate that, feel free to add a response or a few claps, it inspires me to write more.

Nginx Django Python MySQL Ubuntu Server